DevNexus | AgileBridge

CODING STANDARDS

Prepared By

DevNexus

devnexus28@gmail.com

agilebridge

CONSIDERED SOFTWARE SOLUTIONS

CODING STANDARDS

Repository Structure:

At a high level, the repository is organised as follows:

```
├── database
│   ├── init
│   └── scripts
├── docs
│   ├── API Contracts
│   │   ├── Auth
│   │   ├── Listings
│   │   └── University
│   ├── diagrams
│   ├── requirements
│   ├── research
│   └── wireframes
├── infra
│   ├── azure
│   │   ├── modules
│   │   └── parameters
│   └── docker
├── src
│   ├── backend
│   │   ├── Api
│   │   │   ├── BackgroundServices
│   │   │   ├── Controllers
│   │   │   ├── Hubs
│   │   │   ├── Middleware
│   │   │   ├── Notifiers
│   │   │   └── Properties
│   │   ├── Infrastructure
```

```
│   │   ├── AI
│   │   ├── Cache
│   │   ├── Maps
│   │   ├── Messaging
│   │   ├── Notifications
│   │   ├── Payments
│   │   ├── Persistence
│   │   ├── Realtime
│   │   ├── Search
│   │   └── Storage
│   ├── Modules
│   │   ├── Audit
│   │   ├── Chat
│   │   ├── Disputes
│   │   ├── Identity
│   │   ├── Listings
│   │   ├── Notifications
│   │   ├── ReferenceData
│   │   ├── Reputation
│   │   ├── Reservations
│   │   ├── Reviews
│   │   ├── SharedKernel
│   │   ├── Transactions
│   │   └── Wishlist
│   ├── Tests
│   │   ├── Integration
│   │   └── Unit
│   └── Workers
└── web
    ├── playwright-report
```

Backend (src/backend)

The backend is organized into distinct layers, each with a clear responsibility:

- **Api** - the entry point layer. Contains Controllers, SignalR Hubs, Middleware, BackgroundServices, and Notifiers. This is where HTTP/realtime requests come in and responses go out; it should stay thin and delegate real work to Modules.
- **Infrastructure** -cross-cutting technical concerns that support the rest of the application: AI, Cache, Maps, Messaging, Notifications, Payments, Persistence, Realtime, Search, Storage. This is where we talk to external services and shared technical plumbing, kept separate from business logic.
- **Modules** - the business/domain logic, organized by bounded context: Audit, Chat, Disputes, Identity, Listings, Notifications, ReferenceData, Reputation, Reservations, Reviews, SharedKernel, Transactions, Wishlist. This is where domain rules and workflows live.
- **Workers** - reserved for standalone background worker processes, separate from the lightweight hosted services in Api/BackgroundServices.
- **Tests** - split into Unit, Integration, and E2E, matching the frontend split (see Testing Conventions below).

- **What belongs where:** business rules and workflows belong in Modules; anything that talks to an external system, a database, or a technical concern shared across modules belongs in Infrastructure; request/response handling, routing, and real-time endpoints belong in Api.
- **Module internal structure:** each module typically contains a Models folder and a Repository for the data access plus a ModuleException.cs , ModuleService.cs and IModuleService.cs. Some modules incluse additional domain-specific folders.

Frontend (src/web)

- **components** -reusable UI building blocks.
- **pages** -top-level routed views, composed from components.
- **Hooks**-custom React hooks encapsulating reusable stateful logic.
- **services** -API/data access calls.
- **store** -application state management.
- **providers** -React context providers.
- **lib** / **utils** - shared helper code.
- **types** -TypeScript type definitions.
- **Tests**- frontend test suite.

What belongs where: UI rendering and layout belongs in components/pages; reusable stateful logic belongs in hooks; anything talking to the backend API belongs in services; shared, non-React helper logic belongs in lib/utils.

Naming Conventions

Backend

Enforced via . editorconfig as hard CI failures (severity error), checked by dotnet format --verify-no-changes with EnforceCodeStyleInBuild=true:

- Private fields: _camelCase
- Interfaces: I-prefixed, PascalCase (e.g. IChatService)
- Types, methods, properties, events: PascalCase

Not covered by .editorconfig - these remain review-only conventions, enforced by human reviewers rather than tooling:

- **Exceptions**: {Module}Exception suffix (e.g. ChatException).

- **Services**: {Module}Service class implementing I{Module}Service interface (e.g. ChatService / IChatService).

Frontend

- **Hooks**: uses standard useXXX naming convention
- **Services**: {type of service}Service (e.g. authService)

Formatting & Linting

- **Backend**: dotnet format, enforced as a CI failure via dotnet format
- **Frontend**: ESLint.
- **SonarCloud**: static analysis.
- **Gitleaks**: secret scanning.

Logging Conventions

- We use ILogger<T> for logging - no Console.WriteLine calls should be committed.
- ILogger<T> is not yet applied across the entire codebase; it's currently used in critical areas of the system that are prone to errors, with broader adoption expected as the codebase and system expands.

Comment Conventions

- TODO(initials) -for planned follow-up work, tagged with the author's initials.
- FIXME(initials) -for known issues that need addressing, tagged with the author's initials.
- Temporary/debug comments should be removed before merging, they should not be left in reviewed, merged code.

Git Workflow

- **Branch strategy**: separate long-lived frontend and backend branches exist as the base for their respective areas. Contributors branch off the relevant one to do their work, and when a change is complete, it's pushed back to that base branch. From there, changes are merged into dev, and finally into main.

- **Review requirements**:

  • Feature **branches** are lightweight, no fixed approval requirement.
  • dev requires **2 approvals** before merge.
  • main requires **3 approvals** before merge.

- **CI gates**: CI only gates dev and main the feature-branch merges are not CI-gated.

Testing Conventions

Both backend and frontend follow the same three-layer split:

- **Unit** -tests a single file/class/function in isolation.
- **Integration** -tests the flow of a system end-to-end within the codebase without a full browser, e.g. the payment flow or order flow thus verifying multiple components work together correctly.
- **E2E (Playwright)** -tests the full running application, exercising real user flows through the UI.

Definition of Done

A piece of work is considered done when:

1.The changes have been made and integrated.

2.Unit tests have been written and pass.

3.Integration tests have been written and pass.

4.E2E tests have been written and pass (where applicable to the change).

5.Linting passes.

6.SonarCloud passes.

7.Gitleaks passes.

8.CI is green.

9.The required number of review approvals has been met for the target branch (see Git Workflow above).

Dependency Management:

- **NuGet**: packages in the backend
- **npm**: packages in the frontend
- **Docker**: base images

- **GitHub Actions**: workflow versions

Secrets Management

- **Local development** uses local configuration files.
- **Deployment secrets** live in **GitHub Encrypted Secrets** and are injected into the deployment pipeline and application configuration during deployment.
- **Azure Key Vault is intentionally not used** ,this was a deliberate decision, made on the basis that it would be overkill relative to the project's current scale and requirements.
- **Gitleaks** scans the repository to prevent secrets from being committed in the first place.